

Estrutura de Dados 2

Prof. Igor Calebe Zadi
igor.zadi@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo
Campus Catanduva



III. EDs Não-Lineares



III. EDs Não-Lineares

1. Introdução às Estruturas Não-Lineares
2. Conceitos e Terminologia
3. Representações de Árvores
4. Árvores Binárias
5. Percursos em Árvores Binárias
6. Árvores de Busca Binária (BST)
7. Árvores Balanceadas
8. Árvores B e B+
9. Comparações e Aplicações

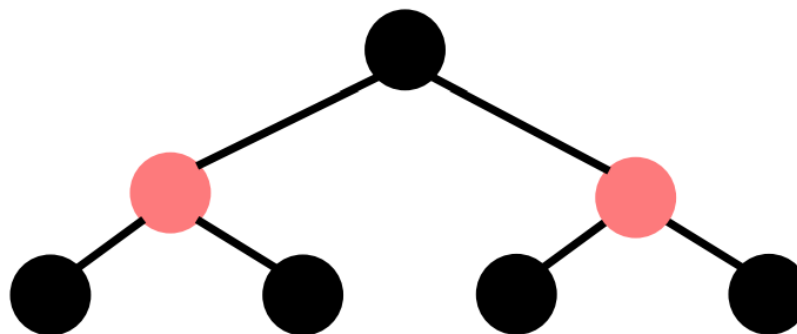


7. Árvore rubro-negra

- Também conhecida como árvore vermelho-preto ou red-black
 - Tipo de árvore binária balanceada
 - Originalmente criada por Rudolf Bayer em 1972
 - Chamadas de Árvores Binárias Simétricas
 - Adquiriu o seu nome atual em um trabalho de Leonidas J. Guibas e Robert Sedgwick de 1978

7. Árvore rubro-negra

- Utiliza um esquema de coloração dos nós para manter o balanceamento da árvore
 - Árvore AVL usa a altura das sub-árvores
- Cada nó da árvore possui um atributo de cor, que pode ser vermelho ou preto
- Além dos dois ponteiros para seus filhos



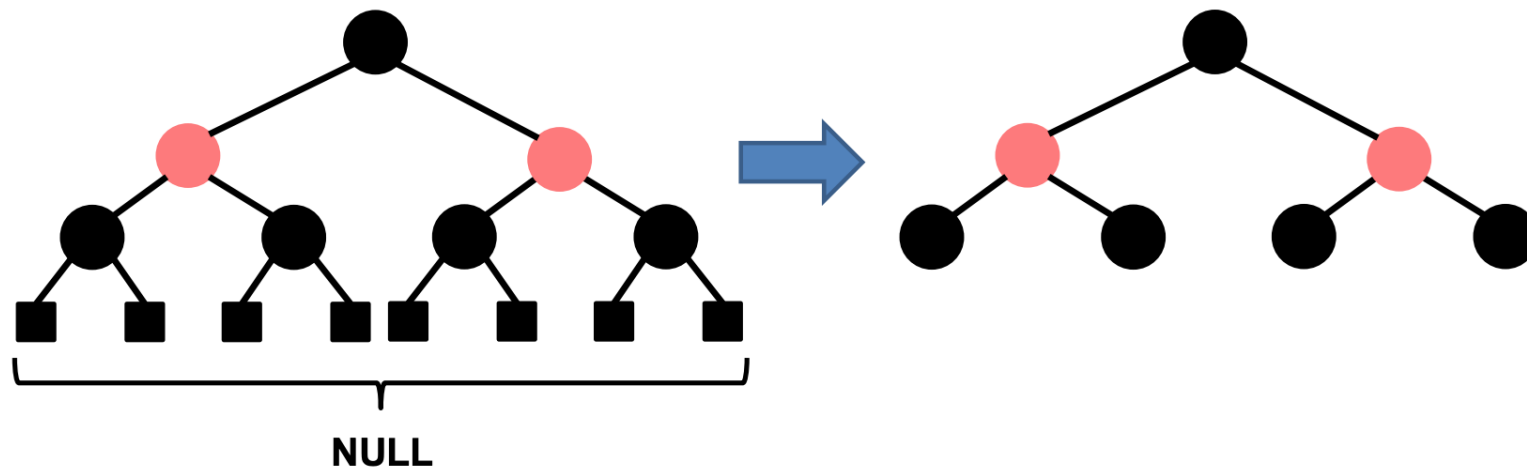


7. Árvore rubro-negra

- Além da cor, a árvore deve satisfazer o seguinte conjunto de propriedades:
 - Todo nó da árvore é **vermelho** ou **preto**
 - A raiz é sempre **preta**
 - Todo nó folha (NULL) é **preto**
 - Se um nó é **vermelho**, então os seus filhos são **pretos**
 - Não existem nós **vermelhos** consecutivos
 - Para cada nó, todos os caminhos desse nó para os nós folhas descendentes contém o mesmo número de nós pretos

7. Árvore rubro-negra

Como todo nó folha termina com dois ponteiros para NULL, eles podem ser ignorados na representação da árvore para fins de didática





7. Árvore rubro-negra - balanceamento

- É feito por meio de **rotações** e **ajuste de cores** a cada inserção ou remoção
 - Mantém o equilíbrio da árvore
 - Corrigem possíveis violações de suas propriedades
 - Custo máximo de qualquer algoritmo é $O(\log N)$



7. AVL x Rubro-Negra

- Na teoria, possuem a mesma complexidade computacional
 - Inserção, remoção e busca: $O(\log N)$
- Na prática, a árvore AVL é mais rápida na operação de busca, e mais lenta nas operações de inserção e remoção
 - A árvore AVL é “mais balanceada” do que a árvore Rubro-Negra, o que acelera a operação de busca



7. AVL x Rubro-Negra

- AVL: balanceamento mais rígido
 - Maior custo na operação de inserção e remoção
 - No pior caso, uma operação de remoção pode exigir $O(\log N)$ rotações na árvore AVL, mas apenas 3 rotações na árvore Rubro-Negra.
- Qual usar?
 - Operação de busca é a mais usada?
 - Melhor usar uma árvore AVL
 - Inserção ou remoção são mais usadas?
 - Melhor usar uma árvore Rubro-Negra



7. AVL x Rubro-Negra

- Árvores Rubro-Negra são de uso mais geral do que as árvores AVL
 - Ela é utilizada em diversas aplicações e bibliotecas de linguagens de programação
 - Exemplos
 - Java: `java.util.TreeMap` , `java.util.TreeSet`
 - C++ STL: `map`, `multimap`, `multiset`
 - Linux kernel: `completely fair scheduler`, `linux/rbtree.h`



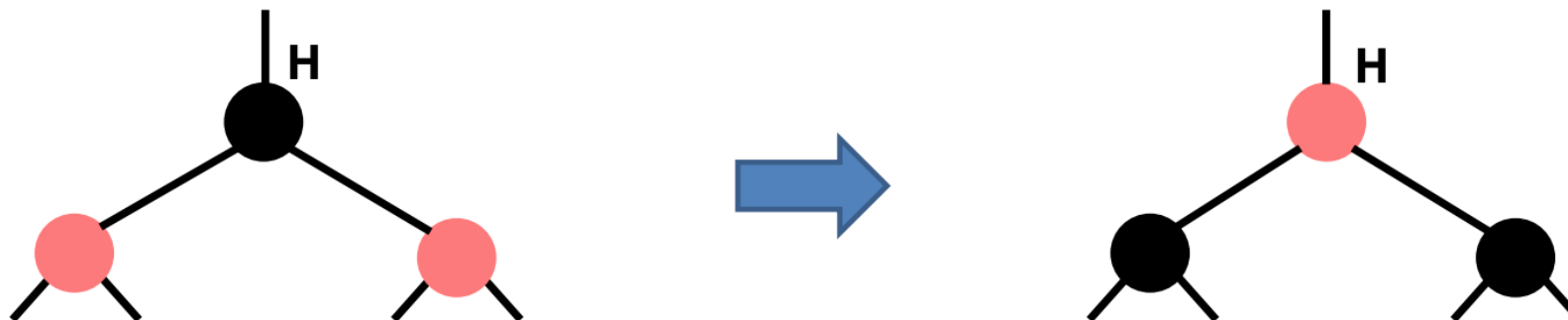
7. Árvore Rubro-Negra

- Definindo uma árvore Rubro-Negra

```
typedef struct No {  
    int valor;  
    int cor; // 1 = vermelho, 0 = preto  
    struct No *esq, *dir;  
} No;
```

7. Árvore Rubro-Negra

- Troca de cores dos nós:
 - Operação de mudança de cor
 - Não altera o número de nós pretos da raiz até os nós folhas.
 - Problema: pode introduzir dois nós consecutivos vermelhos na árvore
 - Deve ser corrigido com outras operações



7. Árvore Rubro-Negra

- Troca de cores:

```
// Troca a cor de um nó e de seus filhos
void trocaCor(No* h) {
    h->cor = !h->cor;
    if (h->esq) h->esq->cor = !h->esq->cor;
    if (h->dir) h->dir->cor = !h->dir->cor;
}
```



7. Árvore Rubro-Negra

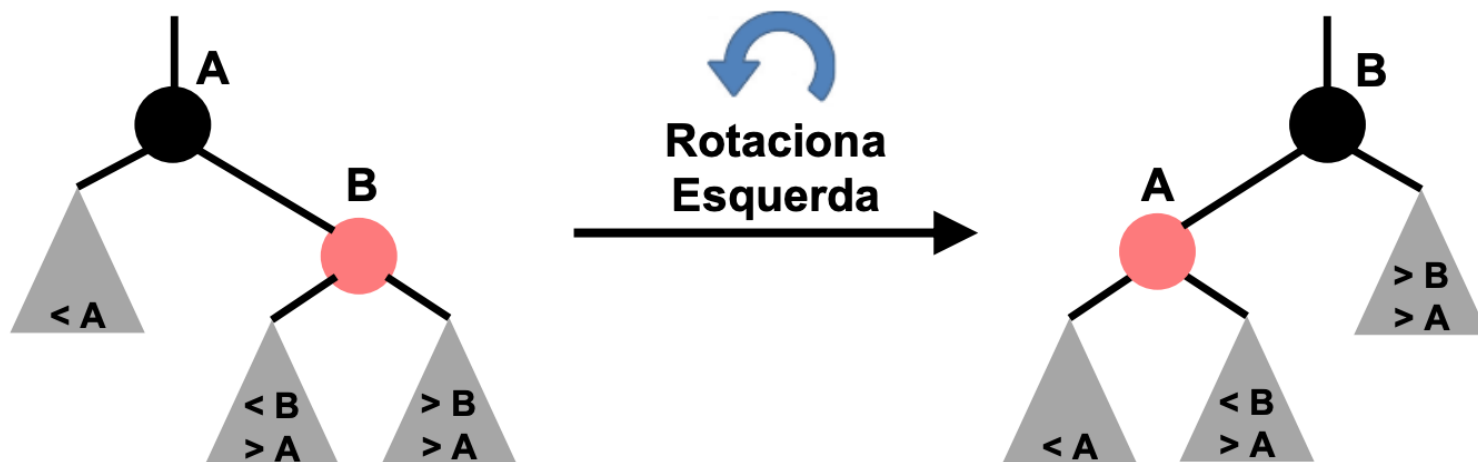
- Rotações
 - Árvore rubro-negra
 - Possui apenas duas funções de rotação
 - Rotação à Esquerda
 - Rotação à Direita

7. Árvore Rubro-Negra

- Funcionamento:
 - Dado um conjunto de três nós, visa deslocar um nó vermelho que esteja à esquerda para à direita e viceversa.
 - Mais simples de implementar e de depurar em comparação com as rotações da árvore AVL
 - As operações de rotação apenas atualizam ponteiros
 - Complexidade é $O(1)$

7. Árvore Rubro-Negra

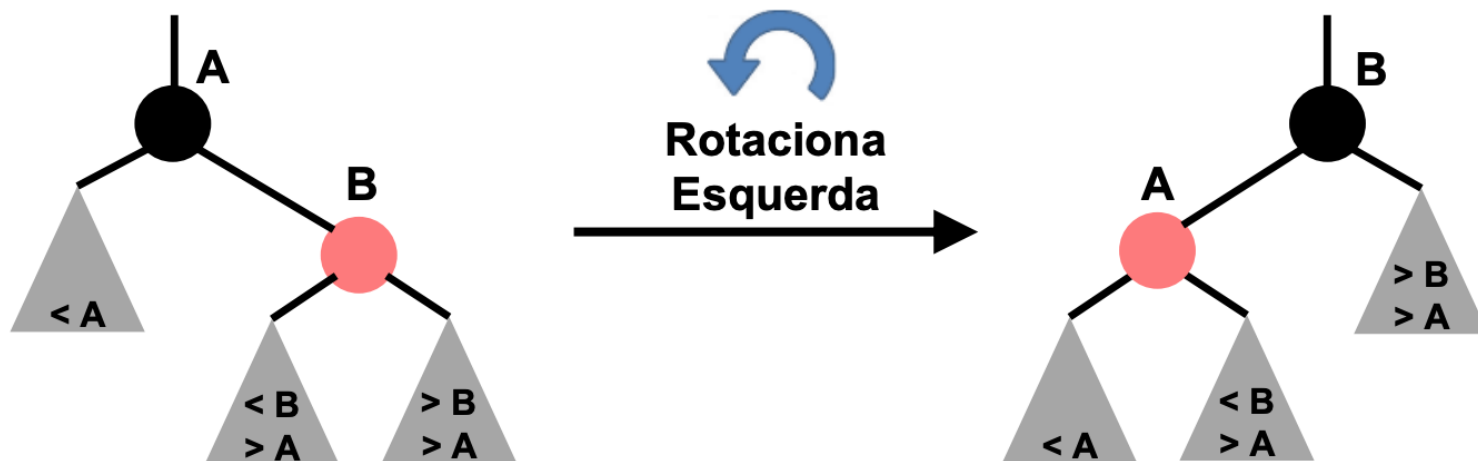
- Rotação à esquerda
 - Recebe um nó A com B como filho direito
 - Move B para o lugar de A, A se torna o filho esquerdo de B
 - B recebe a cor de A, A fica vermelho



7. Árvore Rubro-Negra

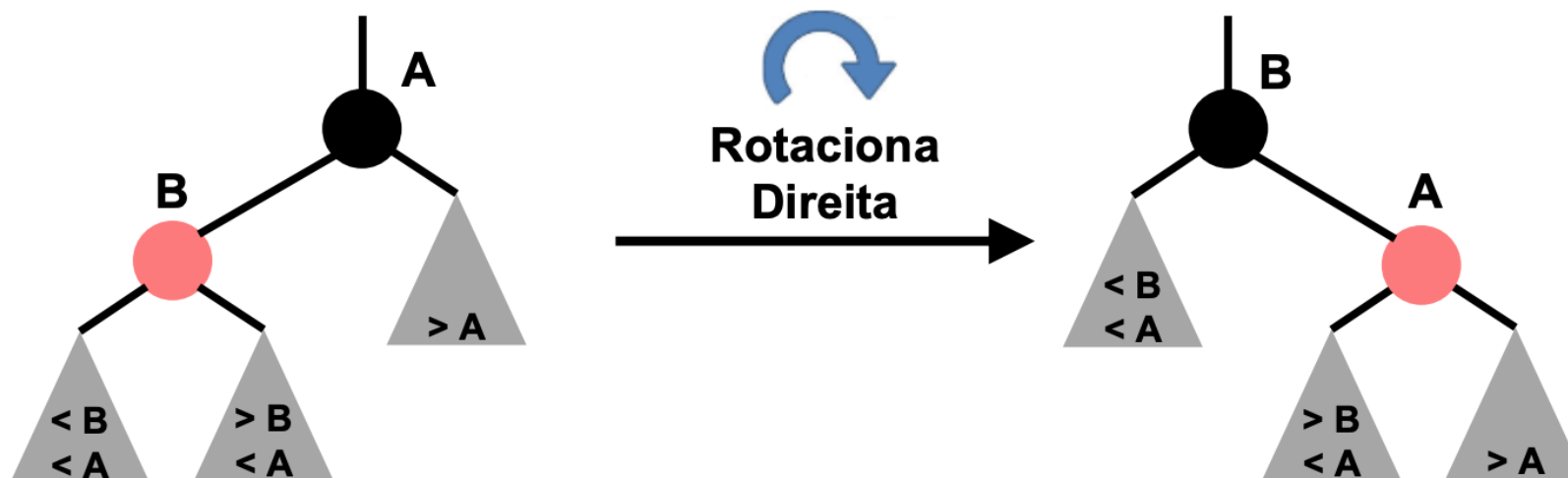
- Rotação à esquerda

```
No* rotacaoEsquerda(No* h) {  
    No* x = h->dir;  
    h->dir = x->esq;  
    x->esq = h;  
    x->cor = h->cor;  
    h->cor = VERMELHO;  
    return x;  
}
```



7. Árvore Rubro-Negra

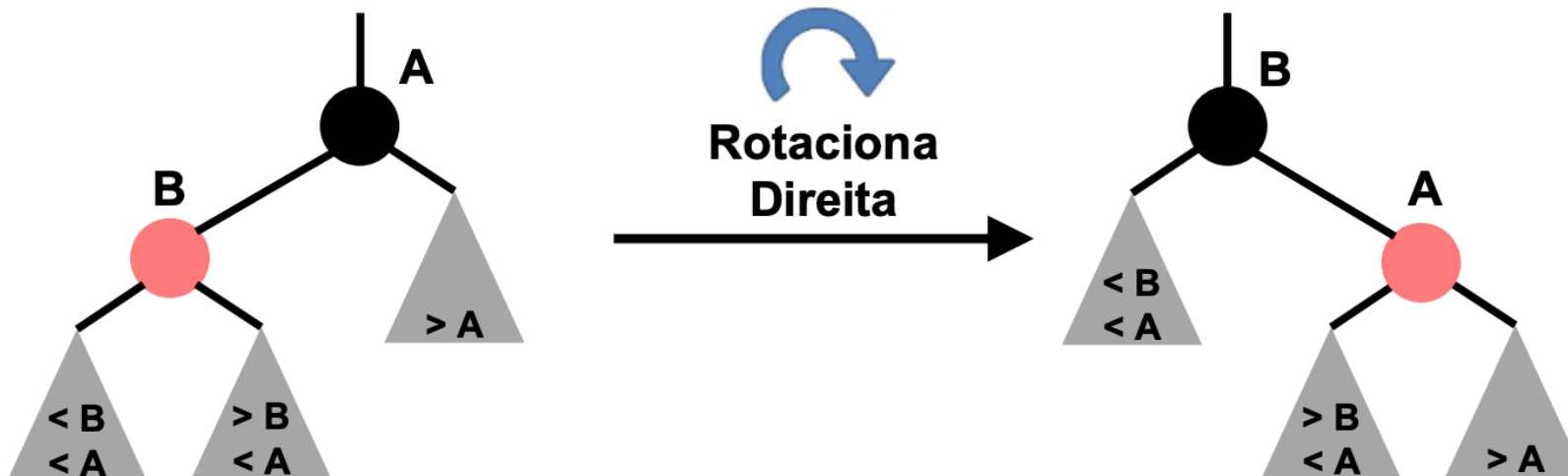
- Rotação à Direita
 - Recebe um nó A com B como filho esquerdo
 - Move B para o lugar de A, A se torna o filho direito de B
 - B recebe a cor de A, A fica vermelho



7. Árvore Rubro-Negra

- Rotação à direita

```
No* rotacaoDireita(No* h) {  
    No* x = h->esq;  
    h->esq = x->dir;  
    x->dir = h;  
    x->cor = h->cor;  
    h->cor = VERMELHO;  
    return x;  
}
```



7. Árvore Rubro-Negra

- Inserção:
 - Similar a inserção na árvore AVL
 - Para inserir um valor V na árvore
 - Se a raiz é igual a NULL, insira o nó
 - Se V é menor do que a raiz: vá para a sub-árvore esquerda
 - Se V é maior do que a raiz: vá para a sub-árvore direita
 - Aplique o método recursivamente
 - Dessa forma, percorremos um conjunto de nós da árvore até chegar ao nó folha que irá se tornar o pai do novo nó



7. Árvore Rubro-Negra

- Todo nó inserido é inicialmente vermelho
- Uma vez inserido o novo nó
 - Devemos voltar pelo caminho percorrido e verificar se ocorreu a violação de alguma das propriedades da árvore para cada um dos nós visitados
 - Aplicar uma das rotações ou mudança de cores para restabelecer o balanceamento da árvore

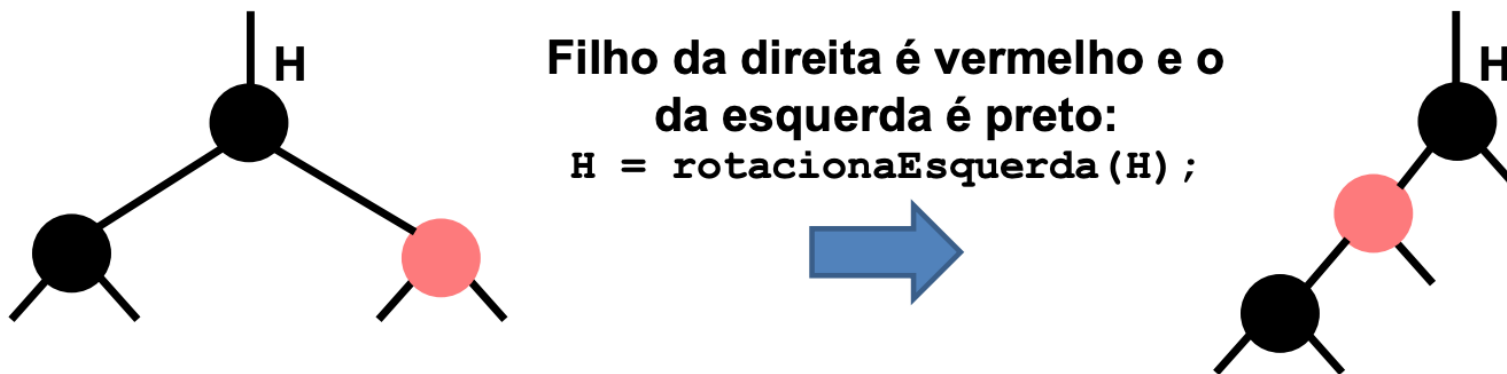
7. Árvore Rubro-Negra

```
No* inserir(No* h, int valor) {  
    if (h == NULL)  
        return novoNo(valor);  
  
    if (valor < h->valor)  
        h->esq = inserir(h->esq, valor);  
    else if (valor > h->valor)  
        h->dir = inserir(h->dir, valor);  
  
    if (ehVermelho(h->dir) && !ehVermelho(h->esq))  
        h = rotacaoEsquerda(h);  
  
    if (ehVermelho(h->esq) && ehVermelho(h->esq->esq))  
        h = rotacaoDireita(h);  
  
    if (ehVermelho(h->esq) && ehVermelho(h->dir))  
        trocaCor(h);  
  
    return h;  
}
```

```
No* novoNo(int valor) {  
    No* n = (No*) malloc(sizeof(No));  
    n->valor = valor;  
    n->cor = VERMELHO;  
    n->esq = n->dir = NULL;  
    return n;  
}
```

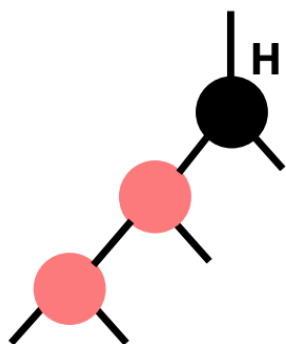
7. Árvore Rubro-Negra

- Violações das propriedades na inserção
 - Filho da direita é vermelho e o filho da esquerda é preto
 - Solução: Rotação à esquerda



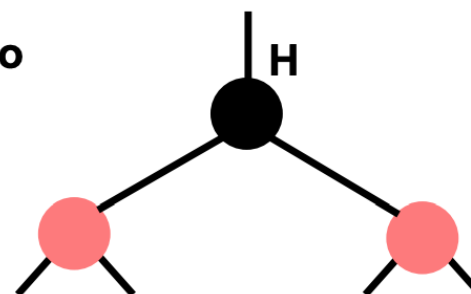
7. Árvore Rubro-Negra

- Violações das propriedades na inserção
 - Filho da esquerda é vermelho e o filho à esquerda do filho da esquerda também é vermelho
 - Solução: Rotação à direita



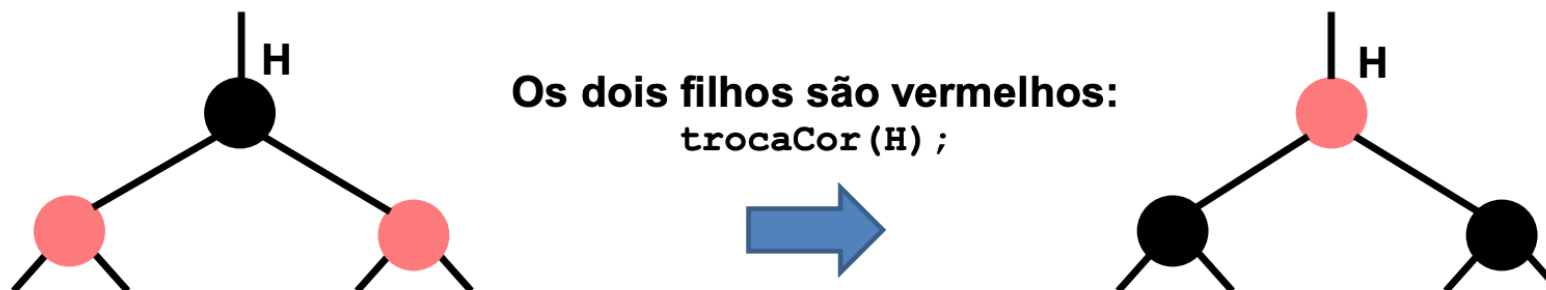
Filho e neto da esquerda são vermelhos:

`H = rotacionaDireita(H) ;`



7. Árvore Rubro-Negra

- Violações das propriedades na inserção
 - Ambos os filhos são vermelhos (é um cuidado)
 - Solução: troca de cores



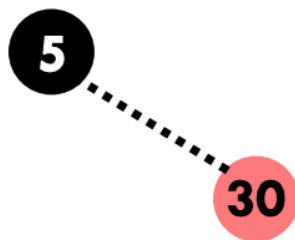
7. Árvore Rubro-Negra

- Inserção (passo a passo):

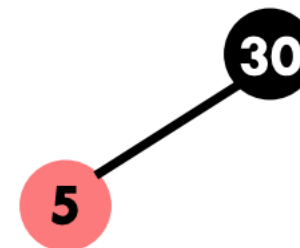
Inserir valor: 1



Inserir valor: 30



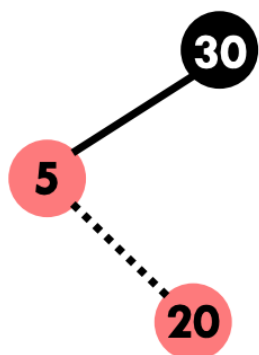
Rotaciona à esquerda
em "5"



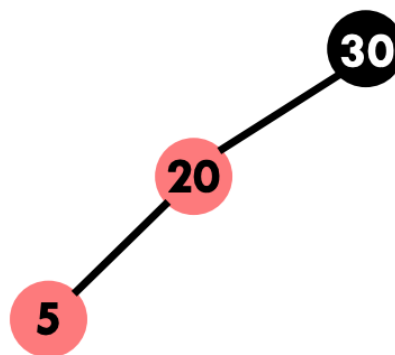
7. Árvore Rubro-Negra

- Inserção (passo a passo):

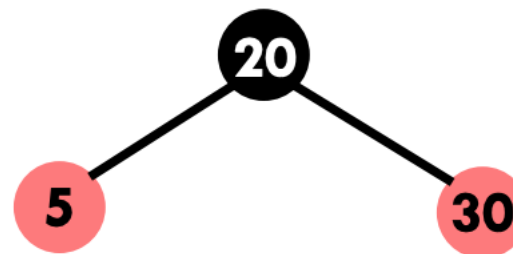
Inserir valor: 20



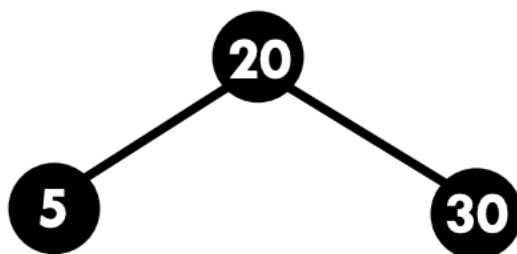
Rotaciona à esquerda
em "5"



Rotaciona à direita
em "30"



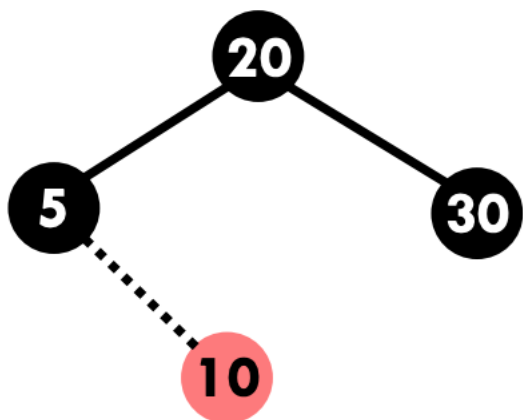
Troca cor em "20"
Raiz fica preta



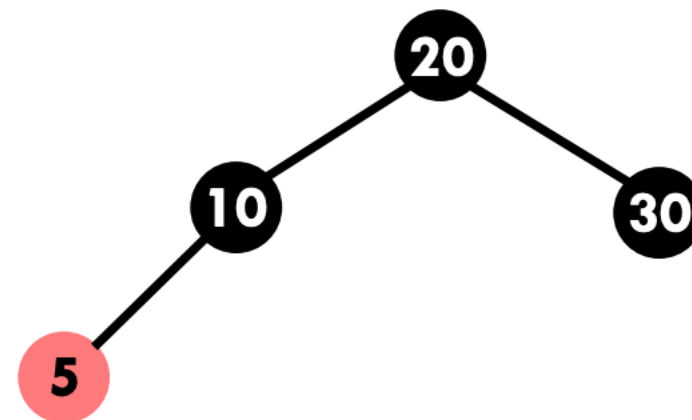
7. Árvore Rubro-Negra

- Inserção (passo a passo):

Inserir valor: 10



Rotaciona à esquerda
em "5"





Observações sobre o material eletrônico

- O material ficará disponível na pasta compartilhada que é acessada sob convite
- O material foi elaborado a partir de diversas fontes (livros, internet, colegas, alunos etc.)
- Alguns trechos podem ter sido inteiramente transcritos a partir dessas fontes
- Outros trechos são de autoria própria
- Esta observação deve estar presente em qualquer utilização do material fora do ambiente de aulas do IFSP - Catanduva